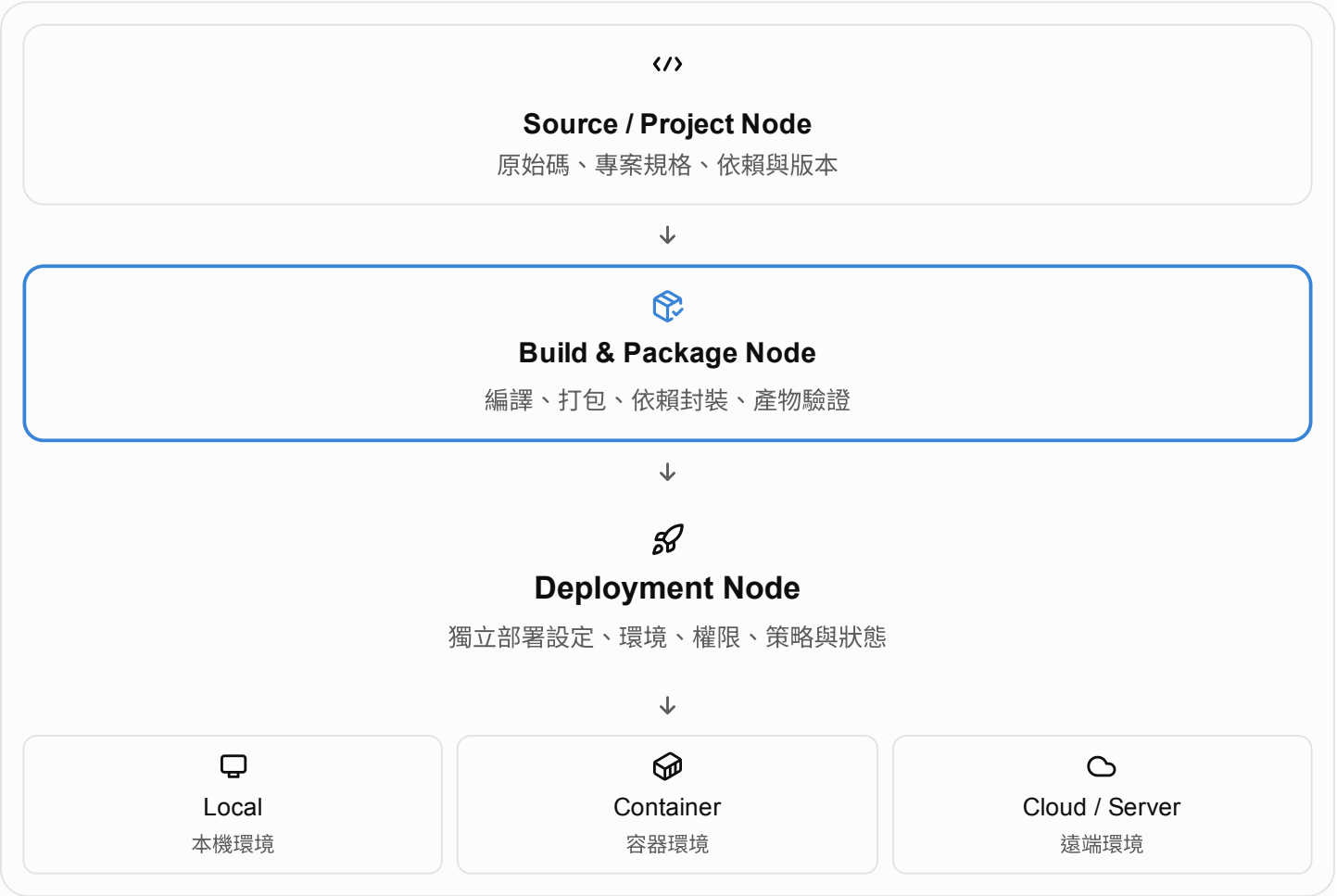
ChatGPT 說：

我認為這個方向很好。對 SGGenAxis 自動開發工具來說，**部署（Deployment）不應只是工作流程最後的一個按鈕，而應該是一種具有獨立屬性、輸入輸出與執行規則的節點（Deployment Node）**。

這樣就能把「開發完成後如何部署」納入整個節點式工程系統，讓不同的部署目標擁有自己的設定、驗證流程與成本控制。

一、Deployment Node 在架構中的位置



這裡要區分兩個概念：

- **Deployment Node Type**：定義這類節點能做什麼、有哪些固定屬性。
- **Deployment Node Instance**：某個專案實際建立的部署節點，例如「部署到測試伺服器」或「打包成 Windows 桌面程式」。

前者是節點的類型規格，後者是使用者在畫布上配置的實例。

二、Deployment Node 的獨有屬性

建議 Inspector 根據部署節點類型，動態顯示專屬屬性，而不是所有節點共用一張塞滿欄位的表單。

Deployment Node

Node Type: deployment

Example Schema

General

節點身分與部署目標

nodeId	節點唯一識別碼
name	使用者自訂名稱
deploymentType	local、container、server、cloud、desktop、web
targetRef	目標環境的資源參照
enabled	是否參與部署流程

Build & Artifact

部署輸入與產物管理

inputArtifact	接收哪個 Build Node 的產物
artifactType	exe、package、container image、web bundle 等
version	部署版本或 Release ID
checksumPolicy	部署前的完整性檢查策略

Environment

部署環境與執行設定

environment	development、staging、production
runtime	執行環境或 Runtime 版本
configProfile	環境設定檔參照
secretRefs	憑證及機密的安全參照，不存明文

Deployment Strategy

部署方法與失敗處理

strategy	replace、rolling、blue-green、canary 等
preDeployChecks	部署前的檢查規則
healthCheck	部署後健康檢查
rollbackPolicy	失敗時的回復規則
approvalPolicy	是否需要人工核准

Execution & Cost

執行控制、用量與結果追蹤

timeoutSeconds	部署工作逾時時間

retryPolicy	重試次數、間隔與錯誤分類
resourceLimits	CPU、記憶體、儲存空間等限制
costBudget	部署任務的成本預算
executionStatus	Idle、Queued、Running、Succeeded、Failed、RolledBack
lastRunRef	最近一次執行紀錄的參照

以上是建議的通用屬性。像 `rolling`、`blue-green` 或 `canary` 這類策略，並非每種部署目標都支援，應由各自的部署 Adapter 宣告可用選項，再由 Inspector 動態顯示。

三、讓每種部署節點有自己的屬性 Schema

這是整體設計最重要的部分。不要用大量 `if deploymentType == ...` 分散在 UI、驗證器及執行程式中；應該建立可擴充的 Node Type Registry。

TypeScript

```typescript
type DeploymentType =
  | "local"
  | "desktop"
  | "web"
  | "container"
  | "server"
  | "cloud";

interface DeploymentNodeDefinition {
  type: DeploymentType;
  label: string;
  schemaVersion: number;

  // 定義 Inspector 欄位及驗證規則
  propertySchema: PropertySchema;

  // 定義可接受的輸入與輸出
  ports: {
    inputs: PortDefinition[];
    outputs: PortDefinition[];
```

這樣的好處是：

1. 新增部署類型時，只要註冊新的 Definition 與 Adapter，不必重寫整個編輯器。
2. Inspector 根據 `propertySchema` 自動產生欄位。
3. 驗證器與執行引擎讀取同一份節點定義，避免 UI 允許的設定和實際執行能力不一致。
4. 節點規格可以版本化，舊專案仍能辨識原本的設定。

四、與降低 Token 成本的關係

部署節點本身不會直接減少 Token，但它可以減少 AI 需要處理的資訊量。

例如，當使用者選擇「部署到 Windows 桌面環境」時：

- 系統直接從節點 Schema 取得必要屬性。
- 確定性程式檢查產物、路徑、版本與必要設定。
- 只有遇到不完整規格、相依性衝突或非預期錯誤時，才呼叫 AI。
- AI 接收的是部署節點、相關 Build Node、必要錯誤資訊，而不是整個專案。

換句話說，**Deployment Node 是可供 AI 查詢的結構化狀態，不是每次都要重新描述給 AI 的一大段 Prompt**。

我會建議下一步把這個概念延伸成通用的 **Node Type System**：每一類節點都有自己的 Schema、Ports、Capabilities、Validation Rules 與 Execution Adapter。如此一來，部署節點只是其中一種，未來 Build、Test、Package、Publish、Monitor 等節點也能沿用相同架構。